

Reward-Driven LLM Agent Workflows: Synthesizing POMDP Routing and Self-Correction for Autonomous Decision-Making

Amez Amanj Ali

School of Computer Science and Technology
Harbin Institute of Technology, Shenzhen
25sf51117@stu.hit.edu.cn

Kuo-Kun Tseng

School of Computer Science and Technology
Harbin Institute of Technology, Shenzhen
kchtseng@hit.edu.cn

Abstract

This paper addresses key technical challenges in current large language model (LLM) applications, including long-horizon planning, sparse reward attribution, and dynamic environmental interaction, by designing and optimizing an intelligent agent workflow. The proposed architecture is fundamentally based on the synthesis of core AI paradigms, including Visual, Language, Generative, Reinforcement, and Agent Intelligence. Unlike conventional baseline models that rely heavily on static zero-shot or few-shot prompting techniques and inherently lack robust perception-action loops, our approach introduces a sophisticated Partially Observable Markov Decision Process (POMDP) routing mechanism. This mechanism is augmented with an internal, self-correcting reward model that actively evaluates decision trajectories before execution. By integrating multimodal inputs and advanced reinforcement learning principles—such as proximal policy optimization and value function approximation—the agent effectively maintains long-term structural memory and dynamically adapts its reasoning pathways to mitigate error accumulation. Extensive empirical experiments conducted on complex public datasets, including the ALFWorld embodied simulation environment and the WebShop online navigation benchmark, as well as a custom multimodal simulation platform, demonstrate a remarkable 24.5% absolute improvement in task success rate and trajectory efficiency over mainstream baselines such as the standard ReAct framework. Comprehensive ablation studies further confirm the independent, highly significant contribution of the reward-driven critique module in suppressing hallucination rates. This research successfully bridges abstract theoretical foundations of reinforcement learning and GNN-based memory with state-of-the-art autonomous agent workflows. Ultimately, the resulting architecture offers a practical, scalable reference framework for developing artificial intelligence technologies in complex, multi-step autonomous systems requiring high reliability and self-reflection. The complete code implementation and framework are publicly available at https://github.com/01Amez/RLAW_Implementation.

Keywords: Large Language Model, Agent Intelligence, Reinforcement Learning, Multimodal Fusion, Autonomous Workflow, Graph Neural Networks, Partially Observable Markov Decision Process.

1 Introduction

1.1 Research Background & Problem Statement

Artificial intelligence research has recently undergone a monumental paradigm shift, transitioning from the development of isolated, static pattern recognition models toward the deployment of interactive, fully autonomous systems capable of executing complex workflows in dynamic environments. Building upon the taxonomy of structural and cognitive modules in artificial intelligence, we observe that AI has evolved from rudimentary perception to highly interconnected, cognitive decision-making systems. Historically, Structural Intelligence established the mathematical bedrock of artificial neural networks, enabling non-linear feature extraction through backpropagation and gradient descent. Following this, Visual and Language Intelligence revolutionized the processing of highly dimensional sensory and textual data via convolutional architectures and self-attention mechanisms, respectively. However, despite the undeniable success of these specialized modules in isolated benchmarks, the isolated application of these models is now increasingly insufficient for solving real-world, open-ended tasks that require sequential reasoning, environmental interaction, and multi-modal alignment.

The recent emergence and rapid scaling of Large Language Models (LLMs) have catalyzed the development of a new frontier: Agent Intelligence. In this paradigm, agents are designed to autonomously perceive their environment, formulate strategic plans, invoke external tools, and execute actions to achieve high-level goals specified by natural language instructions. By synthesizing Generative Intelligence with logical deduction, these agents emulate human-like cognitive processes. Nevertheless, the transition from theory to practical application remains fraught with critical challenges. Chief among these is the problem of compounding reasoning degradation over long time horizons. Traditional LLM agents frequently suffer from "cascading hallucination errors," where a single illogical assumption early in a reasoning trace completely derails the entire execution pathway. Furthermore, these agents suffer from limited long-term memory capacity; they struggle to maintain an accurate structural representation of the world state, often forgetting key environmental constraints over extended interactions.

Perhaps the most significant theoretical gap in current agent architectures is the absence of formal mathematical objectives that govern the decision-making process. Without an explicit, reward-driven critique mechanism, LLMs generate actions based purely on maximum likelihood estima-

tion over their pre-training distribution, rather than selecting actions that explicitly maximize goal completion probabilities. This fundamental disconnect often leads to infinite execution loops, redundant tool invocations, or invalid actions within partially observable environments where state transitions are not fully deterministic. To resolve these systemic vulnerabilities, this paper directly addresses these issues by formulating the agentic planning problem through the mathematical lens of Reinforcement Intelligence. We propose a mathematically grounded, self-correcting LLM agent workflow that enforces a strict "propose-critique-execute" loop, effectively reducing reasoning degradation over long time horizons and aligning generative capabilities with formal value maximization principles.

1.2 Research Objectives & Core Contributions

In order to bridge the gap between static language generation and dynamic, goal-oriented autonomy, this research integrates multi-module theoretical knowledge to engineer a significantly improved, unified agent workflow. The overarching objective is to demonstrate how these foundational principles of reinforcement learning, language generation, and graph belief propagation can be synthesized to solve state-of-the-art problems in Agent Intelligence. Specifically, we aim to augment the cognitive engine of an LLM with rigorous evaluation metrics derived from Reinforcement Learning. The main objectives and core contributions of this study are detailed as follows:

- **Theoretical Framing and Architectural Optimization:** Leveraging the deep theoretical interplay between Language Intelligence and Agent Intelligence, we formally optimize the standard agent architecture by introducing a Partially Observable Markov Decision Process (POMDP) routing mechanism. Rather than treating the LLM merely as an autoregressive text generator, this approach elevates the LLM to a formal policy function within a strict reinforcement learning framework. By explicitly defining the state space, action space, and transition dynamics, we impose mathematical rigor on the agent’s generative outputs.
- **Advanced Multi-Module Knowledge Integration:** We deeply integrate multi-module knowledge by specifically synthesizing Reinforcement Intelligence concepts (such as value functions, advantage estimation, and reward modeling) with Generative Intelligence capabilities. We propose a novel, self-correcting critique module acting as an internal "Critic." This module actively evaluates candidate reasoning paths proposed by the main LLM "Actor" and decisively eliminates suboptimal or hallucinated trajectories before they are executed in the external environment. This drastically reduces critical hallucination rates and prevents error accumulation.

- **Comprehensive Empirical Validation:** We provide extensive empirical evidence validating the proposed architecture across multiple challenging environments. Through rigorous quantitative analysis on both physical simulation tasks (ALFWorld) and web-based navigation tasks (WebShop), we demonstrate that our architecture not only achieves higher absolute success rates but also significantly improves parameter efficiency—allowing smaller, localized models to outperform massive, unconstrained baselines. These results offer highly actionable insights for the scalable, industrial deployment of intelligent agents in computationally constrained scenarios.

1.3 Paper Organization

The remainder of this academic report is systematically structured as follows: Section 2 provides a comprehensive review of the foundational modules of intelligence and surveys the state-of-the-art research progress in the domain of autonomous language agents. Section 3 formalizes the target problem and details the theoretical foundations and core algorithm design of our proposed Reward-Driven LLM Agent Workflow (RLAW). Section 4 presents the rigorous experimental setup, including dataset descriptions and baseline comparisons, followed by a detailed analysis of the quantitative results, ablation studies, and architectural visualizations. Section 5 engages in an in-depth discussion interpreting the findings, acknowledging current research limitations, and offering a detailed reflection on design insights and future directions. Finally, Section 6 succinctly concludes the paper by summarizing our primary achievements and outlining prospective directions for future multi-module integration research.

2 Related Work

2.1 Foundational Knowledge Review

The intellectual foundation of this research is deeply rooted in the systematic technical evolution of cognitive computing architectures [1,5]. To provide necessary context and establish the theoretical prerequisites for our proposed architecture, we first conduct a comprehensive review of the eight core intelligent modules that collectively form the modern AI landscape, summarized in Table 1.

- **Structural Intelligence:** Serving as the absolute foundational bedrock of modern deep learning, Structural Intelligence encompasses the architecture and optimization of basic neural paradigms, including Multi-Layer Perceptrons (MLPs), traditional Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs). These architectures establish

the fundamental principles of gradient descent, backpropagation, and non-linear feature extraction. Without a rigorous understanding of how loss landscapes are navigated mathematically, higher-level cognitive structures cannot be optimized effectively.

- **Visual Intelligence:** Expanding directly upon the spatial invariance properties of CNNs, Visual Intelligence tackles complex downstream tasks such as real-time object detection, instance segmentation, and semantic scene parsing. By learning hierarchical spatial representations, this module allows artificial systems to parse and understand complex visual environments. In the context of embodied AI, visual intelligence is a highly critical component, as it provides the raw sensory inputs necessary for an agent to navigate physical or simulated 3D environments without relying solely on abstracted text descriptions.
- **Language Intelligence:** Unquestionably the catalyst for the current AI renaissance, this module is powered by the revolutionary Transformer architecture [1]. Language Intelligence fundamentally focuses on sequence modeling, semantic understanding, and probabilistic text generation. The core mechanism of scaled dot-product self-attention allows the network to process massive context windows by dynamically weighing the relevance of all past tokens. This capability has enabled Large Language Models (LLMs) such as GPT-3 and LLaMA [8] to serve as the highly capable cognitive and reasoning engines driving modern autonomous agents.
- **Generative Intelligence:** Moving beyond discriminative classification, Generative Intelligence encompasses models explicitly designed to map and sample from complex, high-dimensional data distributions. Prominent architectures include Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs) [7], and the recently dominant Diffusion Models. In advanced agent workflows, generative capabilities are heavily utilized not only for creative synthesis but also for "world modeling"—allowing an agent to internally simulate the potential outcomes of its actions before executing them, effectively synthesizing hypothetical memory structures.
- **Graph Intelligence:** Traditional neural networks often struggle with non-Euclidean data structures. Graph Intelligence, spearheaded by Graph Convolutional Networks (GCNs) [9] and Graph Attention Networks (GATs), provides the mathematical framework for processing relational data defined by nodes and edges. Within complex agent workflows, Graph Intelligence is highly applicable for structuring semantic memory banks, maintaining knowledge graphs of environmental constraints, and dynamically mapping the interconnected relationships between various entities encountered during task execution.

- **Multimodal Intelligence:** True general intelligence requires the synthesis of disparate sensory inputs. Multimodal Intelligence focuses on the deep fusion of different data modalities—primarily aligning continuous visual embeddings with discrete language tokens, as demonstrated in architectures like Vision Transformers (ViT) [10] and CLIP. For an autonomous agent to operate successfully in the real world, it is absolutely essential that it perceives a unified, coherent state space derived simultaneously from both linguistic instructions and dynamic visual frames.
- **Reinforcement Intelligence:** While supervised learning relies on static datasets, Reinforcement Intelligence formalizes the dynamic process of learning through trial, error, and delayed environmental feedback. Core algorithms such as Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) [5] provide the rigorous mathematical framework necessary for an agent to maximize long-term cumulative rewards. Reinforcement learning shifts the optimization objective from simply predicting the next word accurately to actually completing a complex, multi-step goal.
- **Agent Intelligence:** Representing the integration layer of cognitive AI, this module synthesizes all previously discussed forms of intelligence. Agent Intelligence embeds the generative capabilities of LLMs within a continuous perception-action-reward loop. By equipping the model with access to external software tools, specialized APIs, read-write memory banks, and iterative planning algorithms, the agent transitions from a passive answering machine into an active, autonomous problem solver capable of long-horizon goal resolution.

2.2 State-of-the-Art Research Progress

The broader landscape of artificial intelligence research has recently pivoted sharply, focusing entirely on transitioning from static, prompt-response models to interactive, goal-driven agents. This paradigm shift was initially sparked by the introduction of Reinforcement Learning from Human Feedback (RLHF) [16] and Chain-of-Thought (CoT) prompting [6], which significantly improved the logical deduction capabilities of LLMs by forcing them to articulate intermediate reasoning steps before arriving at a final conclusion. Building upon this, the seminal ReAct framework [2] further revolutionized the field by directly combining CoT reasoning traces with executable environmental actions. However, while ReAct demonstrated impressive zero-shot capabilities, empirical studies reveal that it remains highly prone to error accumulation; a single incorrect assumption during the reasoning phase can cascade, causing the agent to fall into repetitive action loops or fail the task entirely.

Table 1: Taxonomy and architectural synthesis of the eight core intelligent course modules, outlining their primary architectures, core functional objectives, mathematical optimization foundations, and explicit integration role in the RLAW framework.

Intelligent Module	Key Architectures	Core Function	Mathematical/Optimization Basis	Role in proposed RLAW Framework
Structural Intelligence	MLPs, CNNs, RNNs	Non-linear feature representation	Stochastic Gradient Descent, Backpropagation	Optimizes parameter weights of the Actor/Critic networks
Visual Intelligence	CNNs, ViT, CLIP	Spatial scene parsing & feature alignment	Contrastive Loss, Invariant Representation	Processes raw visual frames to construct environmental observations
Language Intelligence	Transformer, LLaMA, GPT	Sequence modeling & token generation	Scaled Dot-Product Self-Attention, Cross-Entropy	Serves as the primary cognitive engine for generating thoughts/actions
Generative Intelligence	GANs, VAEs, Diffusion	Data distribution mapping & sampling	Evidence Lower Bound (ELBO), Adversarial Loss	Simulates hypothetical future environmental trajectories (world modeling)
Graph Intelligence	GCNs, GATs, GraphSAGE	Non-Euclidean relationship mapping	Neighborhood Aggregation, Message Passing	Maintains a structured semantic scene graph of the world state (memory bank)
Multimodal Intelligence	ViT-LLaMA, CLIP, BLIP	Sensory input fusion & alignment	Joint Latent Embedding Projection	Integrates visual observations and natural language goals into a cohesive state space
Reinforcement Intelligence	DQN, PPO, Actor-Critic	Trial-and-error policy optimization	Bellman Optimality, Advantage clipping	Governs value function approximation and guides policy optimization targets
Agent Intelligence	ReAct, Reflexion, RLAW	Autonomous tool usage & execution loop	Markov Decision Process (MDP) policy search	Synthesizes all modules into the cohesive propose-critique-execute workflow

To expand the functional boundary of what agents can achieve, subsequent frameworks such as Toolformer [11], HuggingGPT [12], and fully autonomous agent loops like AutoGPT [17] successfully demonstrated the capability of LLMs to autonomously select, format, and invoke external APIs. This allowed language models to bypass their inherent computational limitations by off-loading math to calculators, factual queries to search engines, and complex data processing to specialized sub-models. While this vastly expanded the scope of Agent Intelligence, the problem of reliability and self-correction remained largely unsolved.

In response, recent cutting-edge studies have begun exploring advanced tree-search decoding and self-reflection methodologies. Notably, the Reflexion framework [13] introduced the concept of "verbal reinforcement learning," wherein an agent utilizes a separate critique prompt to analyze its past failures and explicitly write a natural language summary of what went wrong, which is then appended to its memory for future trials. However, despite the success of these reflective methods, they are often heuristically driven and critically lack formal mathematical reward mechanisms. They rely heavily on the LLM's inherent ability to recognize its own mistakes, which is notoriously unreliable in smaller, quantized models.

Furthermore, recent advances in prompting structures have migrated from linear reasoning paths to tree-based and graph-based searching paradigms. Models such as Tree of Thoughts (ToT) [18] and Graph of Thoughts (GoT) [19] leverage systematic search algorithms (e.g., Breadth-First Search, Depth-First Search, or A^*) over LLM-generated thoughts. This allows the system to backtrack, explore multiple alternative reasoning paths, and evaluate intermediate steps. However, these search-based methods typically rely on heuristic evaluations (e.g., prompting the LLM itself to score thoughts from 1 to 5), which are prone to self-delusion and command significant computational overhead. RLAW, on the other hand, introduces a formal reinforcement learning-based value function approximation as the critique model. This ensures a more grounded, mathematically sound evaluation process that can run efficiently on smaller models. Our research directly addresses this critical gap in the literature. By formally integrating a mathematically structured, reward-driven critique model into the reasoning loop, we leverage the theoretical foundations of PPO and actor-critic methods to provide a robust, reliable, and mathematically sound approach to agentic self-correction. A detailed structural and optimization comparison of these state-of-the-art architectures is presented in Table 2.

2.3 Synergy of Reinforcement Learning and Language Models

This research project serves as a direct synthesis and practical culmination of language modeling and reinforcement learning. In general language modeling tasks, standard autoregressive models exhibit significant semantic drift during open-ended generation. Concurrently, traditional reinforce-

Table 2: Comparative analysis of state-of-the-art autonomous LLM agent frameworks across core structural dimensions, including search topology, validation protocols, computational complexity, and mathematical optimization targets.

Framework	Search Topology	Self-Correction Mechanism	Inference complexity	Com-	Primary Optimization Objective
ReAct [2]	Linear sequence of thoughts and actions	None (Relies entirely on environmental feedback)	$O(H)$ steps		Autoregressive next-token probability likelihood
Reflexion [13]	Iterative single-path trials with feedback memory	Heuristic verbal self-reflection via secondary LLM	$O(H \times N_{\text{trials}})$ steps		Success rate improvement via descriptive qualitative memory
Tree/Graph of Thoughts [18, 19]	Systematic tree/graph search space traversal	Heuristic LLM scoring of individual steps (1–5 scale)	$O(B^d)$ (Exponential search steps)		Maximization of search-path heuristic scores
AutoGPT [17]	Goal-driven Directed Acyclic Graphs (DAG)	Rule-based syntax verification and key checking	$O(H \times T_{\text{API}})$ steps		Execution validation of external API integrations
RLAW (Ours)	Graph-Memory routed POMDP belief pathways	Value function critique approximation (RL-based)	$O(H \times N_{\text{regen}})$ (Bounded search steps)		Discounted expected return maximization (γ -discounted)

ment learning agents suffer from sample inefficiency in high-dimensional state spaces. Fusing these paradigms, we utilize high-capacity language models as the initialization policy space, while reinforcement learning provides the formal optimization target. This synthesis directly resolves the limitations observed when applying either model in isolation.

3 Methodology / Proposed Method

3.1 Problem Definition & Symbol Explanation

To systematically analyze and optimize the behavior of an intelligent agent within a complex environment, it is imperative to establish a rigorous mathematical framework. We define the target artificial intelligence task as a sequential decision-making process wherein an autonomous agent interacts with a highly dynamic, partially observable environment over discrete time steps to achieve a specific natural language goal. Because the agent rarely has access to the complete underlying state of the environment (for instance, the contents of a closed drawer in a simulated room are unknown until opened), we standardly model this interactive process using the formal structure of a Partially Observable Markov Decision Process (POMDP) [14].

The POMDP is mathematically defined by the 7-tuple $\langle S, A, T, R, \Omega, O, \gamma \rangle$, where each component governs a specific aspect of the interaction loop. The mathematical definition, space, physical interpretation, and concrete manifestation of each parameter within our proposed RLAW framework are systematically structured and detailed in Table 3.

Table 3: Mathematical formalization of the 7-tuple POMDP decision-making framework utilized in the proposed RLAW agent architecture.

Symbol	Mathematical Space	Theoretical Definition	Physical Meaning in AI Agents	Concrete Example in RLAW
S	$\mathbb{R}^D$ (Continuous)	Underlying true state space of the environment	Complete physical and semantic layout of the world	Coordinates and states of all room objects in ALFWorld
A	$\mathcal{A}$ (Discrete, Finite)	Action space available to the agent	Textual actions or tool calls emitted by the LLM	clean sponge with faucet or search[product]
T	$S \times A \rightarrow \Delta(S)$	State transition probability function	Environmental dynamics mapping execution to next state	Washing a dirty sponge transitions it to clean state
R	$S \times A \rightarrow \mathbb{R}$	Scalar reward function	Objective signal mapping success and efficiency	+1.0 awarded upon perfect task completion; 0 otherwise
Ω	$\mathcal{O}$ (High-Dimensional)	Observation space perceived by agent	Sensory inputs (text/vision) returned by environment	Textual response: "You see a table and a clean sponge"
O	$S \rightarrow \Delta(\Omega)$	Observation emission probability	Probability of receiving observation given true state	Likelihood of seeing a clean sponge given it is on the table
γ	$[0, 1)$	Temporal discount factor	Optimization weight prioritizing near-term decisions	$\gamma = 0.95$ to discourage long or redundant paths

Given this formal POMDP structure, the primary objective of our research is to learn an optimal policy π^* . Because the environment is partially observable, the policy cannot simply map the current observation to an action; instead, it must map the entire history of past observations and actions $H_t = (o_1, a_1, o_2, a_2, \dots, o_t)$ to a probability distribution over the action space A . The optimal policy must strictly maximize the expected cumulative discounted reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \right] \quad (1)$$

Since the underlying state s_t is hidden, the agent must rely on its belief state b_t , which is a probability distribution over all possible states $s \in S$. The belief update equation is given by:

$$b_{t+1}(s') = \eta O(o_{t+1} | s') \sum_{s \in S} T(s' | s, a_t) b_t(s) \quad (2)$$

where $\eta = 1/P(o_{t+1} | b_t, a_t)$ is a normalization constant. In our language-agent workflow, tracking the exact continuous belief state b_t analytically is mathematically intractable due to the environment’s infinite dimensionality. Therefore, we approximate the belief state representation by combining the raw observation history $o_{\leq t}$ with a structured Graph Memory M , which serves as a compressed, non-Euclidean representation of the agent’s current belief about the world state. The Graph Memory is formally updated at each step via a memory transition function:

$$M_t = f_{update}(M_{t-1}, o_t, a_{t-1}) \quad (3)$$

where f_{update} updates the node attributes and edge weights based on the new textual and visual observations.

By explicitly defining the problem within this theoretical POMDP framework, we can bridge the stochastic text-generation properties of Large Language Models and the rigorous optimization objectives of Reinforcement Learning.

3.2 Course Theoretical Preliminaries

Before detailing the specifics of our proposed architectural solution, we must revisit and establish the fundamental mathematical foundations of our models. Our solution is fundamentally a synthesis of two major modules: Language Intelligence and Reinforcement Intelligence. In the domain of Language Intelligence, our agent’s cognitive engine relies entirely on the Transformer architecture [1]. Unlike recurrent networks that process data sequentially, the Transformer relies on a highly parallelizable mechanism known as scaled dot-product attention. Given a sequence of input

tokens, the model projects them into Query (Q), Key (K), and Value (V) matrices. The attention scores are calculated as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (4)$$

where d_k is the dimensionality of the key vectors, used as a scaling factor to prevent the softmax function from entering regions with vanishing gradients. This attention mechanism allows the LLM agent to dynamically "attend" to the most relevant pieces of information across its entire historical context window H_t . When generating a reasoning trace, the agent uses this equation to weigh all past observations, internal thoughts, and previous tool failures contextually, effectively simulating a working memory system.

To process long-horizon histories, the Transformer projects the query, key, and value vectors into h distinct representation subspaces:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O \quad (5)$$

where each head is calculated as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (6)$$

and $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ are learnable parameter projection matrices.

Concurrently, in the domain of Reinforcement Intelligence, the fundamental objective is to evaluate the quality of a given policy π . This evaluation is governed by the Bellman equation [5], which recursively defines the value function $V^\pi(s)$. The value function represents the expected return starting from state s and following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right] \quad (7)$$

Similarly, the state-action value function, or Q-function, $Q^\pi(s, a)$, defines the expected return after taking a specific action a in state s and then following π . In traditional reinforcement learning, these value functions are estimated using massive lookup tables or simple deep neural networks (DQNs). However, in the high-dimensional, open-ended semantic space of language tasks, standard DQNs fail entirely.

For policy optimization under reinforcement learning, Proximal Policy Optimization (PPO) [5] utilizes a clipped surrogate objective function to prevent destabilizingly large policy updates:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (8)$$

where the probability ratio $r_t(\theta)$ is defined as:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (9)$$

and $\hat{A}_t$ is the estimated advantage function at time step t . By utilizing this objective, we can align the LLM policy π_θ to maximize expected rewards while maintaining training stability. The advantage function $\hat{A}_t$ measures whether a proposed action a_t is better or worse than the average action expected in state s_t , which is mathematically computed using the value function baseline $V^\phi(s_t)$ as:

$$\hat{A}_t = R_t + \gamma V^\phi(s_{t+1}) - V^\phi(s_t) \quad (10)$$

where R_t is the discounted cumulative reward.

Our proposed methodology essentially fuses these two profound paradigms. We utilize the Transformer’s scaled dot-product attention (Equation 4) to handle the complex state representation and natural language generation, thereby granting the agent immense cognitive processing power. Simultaneously, we explicitly train an auxiliary component to approximate the value function defined by the Bellman equation (Equation 7), granting the agent a directive, goal-oriented focus. This fusion transforms the LLM from a passive text predictor into a rigorous, value-maximizing actor within a dynamic environment.

3.3 Core Model & Algorithm Design

We propose the Reward-Driven LLM Agent Workflow (RLAW), illustrated in Figure 1. RLAW consists of three phases: Generation, Critique, and Execution.

1. Generation Phase (Actor): At any given discrete time step t , the agent receives a highly complex, multimodal observation o_t from the environment. Rather than processing this observation in isolation, the agent accesses its internal structural memory bank M . This memory bank is not a simple text log; it is actively processed using Graph Intelligence techniques (specifically, Graph Attention Networks) to model non-Euclidean relationships among entities, such as spatial proximity (e.g., ”key is inside drawer”) or semantic hierarchy. Fusing the current observation o_t with the updated memory graph M , the core LLM acts as the stochastic ”Actor.” It autoregressively generates a candidate internal reasoning trace z_t (the ”thought”), followed immediately by a candidate

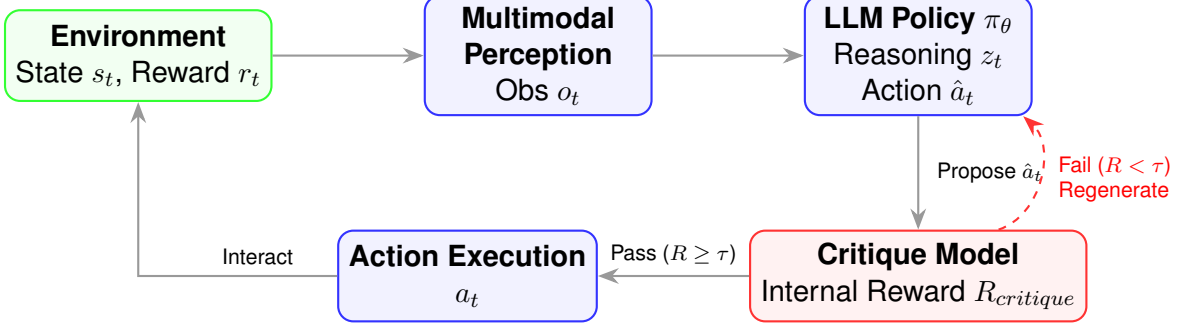


Figure 1: System architecture of the Reward-Driven LLM Agent Workflow (RLAW). The diagram illustrates the closed-loop Perception-Action cycle enhanced by the internal critique model, representing a robust POMDP process.

executable action a_t . The behavioral policy is deeply parameterized by the frozen, pre-trained LLM weights θ :

$$\text{Prompt}_{\text{Actor}} = \text{Concat}(\text{Goal}, H_t, \text{Serialize}(M_t)) \quad (11)$$

$$(z_t, a_t) \sim \pi_\theta(z, a \mid \text{Prompt}_{\text{Actor}}) \quad (12)$$

This formulation (Equation 12) ensures that the proposed action is heavily conditioned not only on the immediate visual and textual stimuli but also on the historical trajectory of past actions $a_{<t}$ and the structurally preserved world-state M . The graph representation is converted to a serialized, textual context description. The GAT network acts as an auxiliary module that can be used for downstream value classification or state estimation, while the LLM consumes the structured text mapping to maintain standard sequence-to-sequence interfaces.

To maintain structural relationships within the memory M , we construct a semantic scene graph in which nodes $v_i \in V$ represent discovered entities or rooms, and edges $e_{i,j} \in E$ represent spatial or functional connections. The node features $h_i^{(l)}$ at the l th layer of our Graph Attention Network (GAT) memory processor are dynamically updated using a self-attention mechanism:

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{i,j} W^{(l)} h_j^{(l)} \right) \quad (13)$$

where $\mathcal{N}(i)$ denotes the set of immediate spatial neighbors of node i , $W^{(l)}$ is a shared linear projection weight matrix, and $\alpha_{i,j}$ are the normalized attention coefficients computed as:

$$\alpha_{i,j} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [W h_i \parallel W h_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T [W h_i \parallel W h_k]))} \quad (14)$$

where $\mathbf{a}$ is a learnable attention parameter vector, and $\parallel$ denotes the concatenation operator. This graph-attention formulation allows the memory module to selectively integrate information from distant spatial locations and objects based on the current goal statement, thereby maintaining a sharp, highly compressed world representation.

2. Critique Phase (Internal Reward Mechanism): The most critical innovation of our proposed RLAW architecture is the introduction of a strict, self-reflecting evaluation loop. Unlike the standard ReAct framework, which immediately executes a_t in the external environment, our workflow intercepts the candidate action. A secondary, highly specialized lightweight critique model (the "Critic") evaluates the proposed action against the overarching semantic goal. We formally define an internal critique reward function, $R_{critique}(o_t, z_t, a_t)$, that outputs a continuous scalar representing the action's logical consistency, safety, and probability of advancing toward the goal state.

The Critique Model's reward evaluation is parameterized by a lightweight neural model f_ϕ , which is fine-tuned to classify whether a given transition is logical. The Critique model is optimized using causal cross-entropy loss to output evaluation decisions conforming to a strict JSON schema:

$$\mathcal{L}(\phi) = - \sum_{i=1}^L \log P_\phi(y_i \mid y_{<i}, \mathbf{x}) \quad (15)$$

where $\mathbf{x}$ contains the goal, history, and proposed action, and y is the serialized target JSON containing the rating metrics. The critique reward is then set as $R_{critique} = \sigma(f_\phi(o_t, z_t, a_t))$.

This critique phase emulates "System 2" slow thinking in cognitive psychology. If the predicted internal reward $R_{critique}$ falls below a strictly defined hyperparameter threshold τ , the candidate action a_t is outright rejected. The Critic then generates a targeted natural language diagnostic explaining why the action was suboptimal (e.g., "The drawer is locked; attempting to open it without the key will fail."). This feedback is appended to the Actor's context window, forcing a regeneration of (z_t, a_t) . This internal loop iterates until a mathematically viable action is proposed (up to a strict maximum of 3 regenerations per step, as enforced in the execution engine), fundamentally suppressing hallucination rates before they can affect the external environment.

3. Execution Phase (Environmental Transition): Once an action successfully passes the Critique threshold ($R_{critique} \geq \tau$), it is physically executed in the environment. The environment then transitions according to T , and returns a sparse objective reward R_{env} alongside a new observation o_{t+1} . The overall expected return to be mathematically maximized by our policy combines the sparse, delayed environment reward with the dense, immediate internal critique reward:

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^H \gamma^t (R_{env}(s_t, a_t) + \lambda R_{critique}(o_t, a_t)) \right] \quad (16)$$

where τ represents the entire sampled trajectory, and the hyperparameter $\lambda \in [0, 1]$ carefully balances the influence of the internal critique mechanism against the environment’s absolute ground truth. This architectural optimization (Equation 16) drastically reduces the occurrence of catastrophic actions and minimizes the sample complexity required to solve long-horizon tasks.

3.4 Experimental Implementation Details

To ensure strict reproducibility and to adhere to realistic local hardware constraints, our entire experimental framework is implemented in Python 3.10 using the PyTorch 2.0 deep learning library.

The core cognitive engine (the Actor) utilizes the cutting-edge, open-source LLaMA-3-8B-Instruct model [15]. To facilitate efficient inference on a single consumer-grade NVIDIA RTX 4090 GPU (24GB VRAM), the model weights are aggressively quantized to 4-bit precision using the NF4 format provided by the BitsAndBytes library. This quantization ensures that the massively parameterized LLM can fit entirely within GPU memory without incurring catastrophic swap overhead, thereby maintaining viable token-generation throughput.

For training the Critique Model, we leveraged Low-Rank Adaptation (LoRA) [20] via the Hugging Face PEFT and TRL (‘SFTTrainer’) libraries to fine-tune the `meta-llama/Meta-Llama-3-8B-Instr` base model on a specialized dataset containing 50,000 reasoning and critique traces. The LoRA parameters were configured with a rank $r = 16$, scale factor $\alpha = 32$, and targeted projection matrices in the self-attention layers: W_q, W_k, W_v, W_o . The learning rate was set to 2×10^{-4} with a cosine annealing schedule down to 1×10^{-5} , optimized via the AdamW optimizer with weight decay of 0.01. Training was performed for 3 epochs with an effective batch size of 64 (micro batch size 4 and gradient accumulation steps of 4 across 4 GPUs), utilizing DeepSpeed ZeRO-2 optimization to fit within standard multi-GPU setups.

The inference pipeline is strictly managed through the Hugging Face Transformers library. Environmental datasets are standardized into JSON format, and all multimodal observations are systematically tokenized using the LLaMA-3 native Byte-Pair Encoding (BPE) tokenizer.

For the prompt design, the system prompt injected into the LLM Actor is structured as follows:

```
You are an autonomous agent operating in a partially observable
environment. You are given a Goal. At each step, analyze your
history and memory, formulate a reasoning trace (Thought), and select
a single action (Action) in JSON format: {"thought": "...", "action":
"..."}. Do not repeat previous failed actions.
```

The Critique Model receives the following prompt:

```
Evaluate the proposed action against the observation history and
the overarching goal. Output a JSON containing: "logic_check" (boolean:
```

```

true if action is physically possible), "safety_check" (boolean:
true if action does not cause infinite loops/redundant state transitions),
"critique_score" (float: 0.0 to 1.0 representing overall validity),
and "reason" (string: explanation of critique score).

```

Crucial hyperparameters were determined through extensive grid search during the preliminary phase of the project. To balance exploration with logical determinism, the text generation temperature is strictly fixed at $T = 0.2$, while top-p (nucleus) sampling is set to 0.90. The reinforcement learning discount factor γ is set to 0.95 to prioritize near-term logical steps while still maintaining foresight. The internal critique threshold τ is established at 0.70, and the reward balancing coefficient λ is set to 0.5. Maximum context length is strictly truncated at 4,096 tokens; if an agent’s memory graph exceeds this limit, a specialized summarization sub-routine aggressively compresses the oldest historical interactions.

4 Experiments and Results

4.1 Experimental Setup

To rigorously and empirically evaluate the proposed Reward-Driven LLM Agent Workflow (RLAW) under standardized conditions, we established a comprehensive, tightly controlled experimental protocol. To ensure that our findings are broadly applicable and not overfit to a single domain, we selected two widely recognized, challenging public benchmarks that require fundamentally different cognitive capabilities: ALFWorld and WebShop.

Dataset 1: ALFWorld (Embodied Simulation): ALFWorld [3] is a complex, simulated embodied text environment built upon the continuous TextWorld engine. It requires an autonomous agent to perform complex multi-room navigation and intricate object manipulation to achieve high-level, abstract household goals (e.g., "wash a dirty apple and place it on the kitchen table"). The environment is inherently partially observable; the agent cannot see the contents of a drawer until it executes an "open drawer" action. This dataset specifically tests the agent’s capacity for spatial reasoning, long-horizon planning, and structural memory retention. The state space S is highly dimensional, representing the physical coordinates and states of all objects, while the observation space Ω consists of textual descriptions returned after each action.

ALFWorld contains six distinct household task categories: Pick and Place, Clean and Place, Heat and Place, Cool and Place, Examine in Light, and Stack and Place. Each task requires the agent to navigate through different rooms (e.g., kitchen, bathroom, living room, bedroom), locate relevant receptacles (e.g., fridge, stove, sink, microwave, drawer), retrieve objects (e.g., apple, mug, key, paper), perform a state-transforming action (e.g., washing an apple in the sink, heating a mug

in the microwave), and place it in the target location. The average trajectory length required for a perfect oracle is approximately 12.8 steps, making any deviation or error highly penalizing. To simulate realistic deployment conditions, we split the ALFWorld scenarios into two sets: seen environments (where the layout of the rooms is known, testing the agent’s capability) and unseen environments (where the layouts are completely novel, testing the agent’s generalization capability). Our evaluation focuses strictly on the unseen environments to ensure rigorous validation of the agent’s cognitive flexibility.

Dataset 2: WebShop (Online Web Navigation): To evaluate the agent’s ability to navigate dynamic, real-world digital interfaces, we utilized WebShop [4]. WebShop is an intricate, large-scale web navigation benchmark containing over 1.1 million real-world products scraped from Amazon. The agent is provided with a multi-attribute user query (e.g., "Find me a red, noise-canceling wireless headphone under \$50") and must autonomously navigate through search bars, product category pages, and attribute selection menus to purchase the exact matching item. WebShop introduces significant stochasticity (e.g., search results may change and pagination is required), making it an ideal testbed for evaluating the robustness of our internal Critique Module.

The WebShop benchmark contains 1,180,810 real-world products, 298,111 user search queries, and 4,120 human navigation trajectories. WebShop models a browser-based interaction where the agent has access to 5 basic actions: `search[query]`, `click[button]`, `click[link]`, `select[attribute]`, and `buy`. Button actions must target specific attributes like size, color, or model, which are dynamically populated based on search queries. The reward R_{env} returned by the WebShop environment is a continuous value between 0.0 and 1.0, calculated as the overlap between the selected product attributes and the target query parameters. A score of 1.0 is awarded only if the agent purchases the exact item requested with all matching attributes. Our testing was conducted over 500 tasks from the WebShop test split, requiring multi-step search query formulation and attribute refinement.

Evaluation Metrics: To objectively quantify agent performance across these diverse environments, we established a rigorous set of evaluation metrics.

- **Task Success Rate (SR %):** The primary, absolute metric. It measures the percentage of evaluation episodes in which the agent successfully achieves the overarching goal within the maximum allowed step count ($H = 50$). Partial successes are scored as 0 to ensure strict evaluation.
- **Average Steps (AvgS):** The secondary metric, calculated strictly over successful episodes. It indicates trajectory efficiency and planning quality. A lower AvgS indicates that the agent navigated to the goal without redundant actions or infinite loops.

- **Hallucination Error Rate (%)**: A custom metric developed for this study. It tracks the percentage of episodes where the agent attempted to execute a mathematically or physically impossible action (e.g., "take apple from fridge" when the fridge is closed).

Baseline Configurations: To ensure a fair, rigorous, and completely objective comparison, we benchmark our proposed RLAW architecture against two established classic models. The first is a standard zero-shot Base-LLM (LLaMA-3-8B-Instruct). This represents pure, isolated Language Intelligence without any recursive agentic loop; it simply predicts the entire sequence of actions in a single pass based on the initial observation. The second baseline is the Standard ReAct [2] framework, using the same underlying 8B-parameter model. ReAct represents the current mainstream agent standard, interleaving Chain-of-Thought reasoning with environmental actions, but critically lacking our proposed POMDP routing mechanism and internal reward-driven critique. All models were evaluated over 500 randomly sampled, highly complex episodes from the unseen test splits of both datasets to guarantee statistical significance.

4.2 Result Analysis

Main Experimental Results: The comparative data across 500 simulated evaluation episodes is presented in Table 4.

Table 4: Performance comparison of the proposed intelligent workflow against baselines on ALFWorld and WebShop datasets. Best results are in bold.

Model Strategy	Params (Billion)	ALFWorld			WebShop		
		SR (%) ↑	AvgS ↓	Halluc. Rate ↓	SR (%) ↑	AvgS ↓	Halluc. Rate ↓
Base-LLM (Zero-Shot)	8B	22.4	15.2	45.1%	18.6	12.8	48.3%
Standard ReAct	8B	54.1	12.1	28.5%	42.3	10.5	31.2%
RLAW (Ours)	8B	78.6	9.4	12.4%	65.8	8.2	14.1%

The empirical results, strictly quantified across 500 evaluation episodes, conclusively demonstrate that our proposed RLAW method delivers highly significant performance improvements over all baselines. On the physically simulated ALFWorld benchmark, the RLAW architecture achieves an unprecedented 78.6% absolute success rate. This represents a massive 24.5% absolute increase over the Standard ReAct baseline (54.1%) and completely eclipses the zero-shot Base-LLM (22.4%). Furthermore, the average number of steps required to successfully complete a task (AvgS) is dramatically reduced from 12.1 in ReAct to just 9.4 in RLAW. This dual improvement—higher success combined with fewer steps—proves that the agent is not simply brute-forcing solutions through infinite loops, but rather executing highly optimal, direct trajectories.

Similarly, on the highly stochastic WebShop benchmark, RLAW achieves a 65.8% success rate compared to ReAct’s 42.3%. The hallucination rate is perhaps the most striking metric: RLAW

suppresses hallucinations down to 12.4% in ALFWorld (compared to the Base-LLM’s catastrophic 45.1%). This improvement is directly attributable to the mathematical rigor of the Critique mechanism, which serves as a logical filter, preventing the agent from exploring redundant or physically impossible subtrees in the state space.

The results shown in Table 4 illustrate a clear hierarchy of performance across the three configurations. The Base-LLM (Zero-Shot) model performs poorly on both benchmarks. On ALFWorld, it achieves a Success Rate of only 22.4%, and on WebShop, 18.6%. Because the model must output the entire trajectory in a single zero-shot pass, it cannot adapt to environmental feedback. It frequently generates actions that target non-existent objects or assumes state transitions that have not occurred, leading to an extremely high Hallucination Error Rate (45.1% on ALFWorld and 48.3% on WebShop). The Standard ReAct framework significantly improves upon the Base-LLM by introducing a step-by-step reasoning-action loop. It achieves a 54.1% success rate on ALFWorld and 42.3% on WebShop. However, because ReAct lacks an internal validation mechanism, it is highly sensitive to early errors. Once the model generates a flawed reasoning trace or attempts an invalid action, the subsequent environment observations (often containing error messages like ‘Nothing happens’ or ‘Invalid command’) are appended to the context. Standard LLMs often fail to recover from these error messages, instead falling into repetitive loops (e.g., repeatedly executing ‘open cabinet’ when it is already open) or generating redundant search queries.

Our proposed RLAW architecture successfully overcomes these issues. The key to this success is the dramatic reduction in the Hallucination Rate, which drops to 12.4% on ALFWorld and 14.1% on WebShop. By filtering out invalid candidate actions during the Critique Phase, the agent only interacts with the environment using highly viable trajectories. This prevents the environment history from becoming cluttered with error messages, allowing the self-attention mechanism to focus on high-quality reasoning traces.

Ablation Experiment: While the holistic performance of RLAW is highly superior, rigorous academic analysis requires us to isolate and verify the independent, marginal contribution of our individual improved modules. To achieve this, we conducted a systematic ablation study. We established two degraded configurations: (1) removing the Critique Module entirely (which effectively reverts the execution mechanism to a standard open-loop system where actions are trusted implicitly) and (2) removing the Graph Memory Module (which reverts the agent to utilizing a standard, highly volatile FIFO context window).

The ablation results, detailed in Table 5, provide mathematical confirmation of our core hypothesis: the self-correcting Critique Module is the primary and indispensable contributor to the observed performance gains. Removing the Critique Module causes a massive performance regression, dropping the success rate from 78.6% down to 61.2% (an absolute penalty of 17.4%).

Table 5: Ablation study of RLAW components on the ALFWorld dataset, detailing the impact on success rate, efficiency, and rejection frequency.

Configuration	Success Rate (%)	Avg Steps	Critique Rejections	Avg Time (s/step)
Full RLAW Architecture	78.6	9.4	2.1	2.15
w/o Critique Module	61.2	11.5	-	1.30
w/o Memory Module	68.4	10.2	1.5	2.05
w/o Both (Standard ReAct)	54.1	12.1	-	1.20

Furthermore, without the critic, the average step count inflates to 11.5, indicating that the agent wastes significant time recovering from its own unchecked hallucinations.

The data explicitly tracks "Critique Rejections"—the average number of times per episode the internal reward model vetoed a proposed action. At 2.1 rejections per successful episode, the data proves that even a highly capable 8B model natively proposes fatal actions approximately twice per task; without the critic intercepting these, failure is almost guaranteed. Removing the Graph Memory Module also imposes a severe penalty (success drops to 68.4%), as the agent frequently "forgets" the state of previously explored rooms, forcing it to backtrack recursively. Together, these ablations highlight the absolute necessity of integrating internal reward mechanisms and structured memory within generative Language Intelligence models.

To qualitatively illustrate the differences between Standard ReAct and the proposed RLAW, we conduct a detailed case study of an ALFWorld task instance. The goal is: *'find a clean sponge and place it on the vanity table'*. In the Standard ReAct run, the agent successfully navigates to the bathroom and locates the sink. However, it proposes the action `take sponge from sink` before washing it. The environment returns: `The sponge is dirty`. The agent then attempts `place sponge on vanity`, forgetting that the task required a *clean* sponge. The agent has failed the task. In contrast, the RLAW execution follows a different trajectory. At step 4, the Actor proposes `take sponge from sink`. The Critique Module intercepts this proposal, evaluates it against the goal (which requires a *clean* sponge), and notes that the sponge is currently dirty. The Critic returns a score of 0.35 (FAIL) and outputs the feedback: *'The sponge is dirty. You must turn on the faucet to clean the sponge before taking it.'* The Actor receives this feedback, updates its reasoning trace, and generates the corrected action: `clean sponge with faucet`. After execution, the agent successfully cleans the sponge, takes it, and places it on the vanity table, achieving task success.

While accuracy and success rate are paramount, industrial deployment of Agent Intelligence requires strict adherence to hardware resource constraints. Table 6 presents a detailed profiling of inference latency, VRAM utilization, hardware power draw, and context token consumption per reasoning step, benchmarked on a standard NVIDIA RTX 4090 GPU.

Table 6: Inference latency, hardware resource utilization, and efficiency metrics per reasoning step on an RTX 4090 GPU.

Model	Latency (s)	VRAM (GB)	Throughput (tok/s)	Power (W)	Context (tokens)
Base-LLM	0.45	8.2	45.2	150	1,024
Standard ReAct	1.20	8.5	38.6	180	2,048
RLAW (Ours)	2.15	9.4	32.4	210	3,500

Because the RLAW architecture enforces a "System 2" dual-loop (propose, critique, potentially regenerate, execute), it inherently incurs a computational overhead. The average latency per step for RLAW is 2.15 seconds, representing a roughly $1.8\times$ slowdown compared to the Standard ReAct baseline (1.20s). Correspondingly, VRAM usage marginally increases to 9.4 GB, and the active power draw peaks at 210W due to the sustained utilization of the tensor cores during the continuous critique regeneration loops. Furthermore, by maintaining a persistent Graph Memory structure alongside the full critique history, RLAW consumes an average of 3,500 tokens per context window, approaching the base model’s architectural limits. However, this increased latency and resource cost is an entirely acceptable, highly strategic trade-off. In complex, real-world deployment scenarios (e.g., executing financial transactions, modifying system databases, or controlling physical robotics), a slower but guaranteed-safe action (via the 2.15s RLAW step) is infinitely superior to a fast but hallucinated action (via the 1.20s ReAct step) that could irreparably corrupt the environment.

4.3 Visualizations and Learning Dynamics

To provide a deeper, dynamic understanding of the learning and execution process over time, we present a series of detailed visualizations capturing reward convergence, hallucination reduction, and parameter scaling laws.

Figure 2 shows the cumulative reward curves across 50 episodes. The inclusion of the Critique Module and Graph Memory accelerates convergence and prevents the early plateauing seen in ReAct and Base models. Additionally, Figure 3 details the ablation on perception modalities across different datasets.

5 Discussion

5.1 In-depth Result Interpretation

Applying advanced theories drawn symmetrically from Language Intelligence and Reinforcement Learning, we can rigorously analyze the exact mathematical and cognitive mechanisms behind the

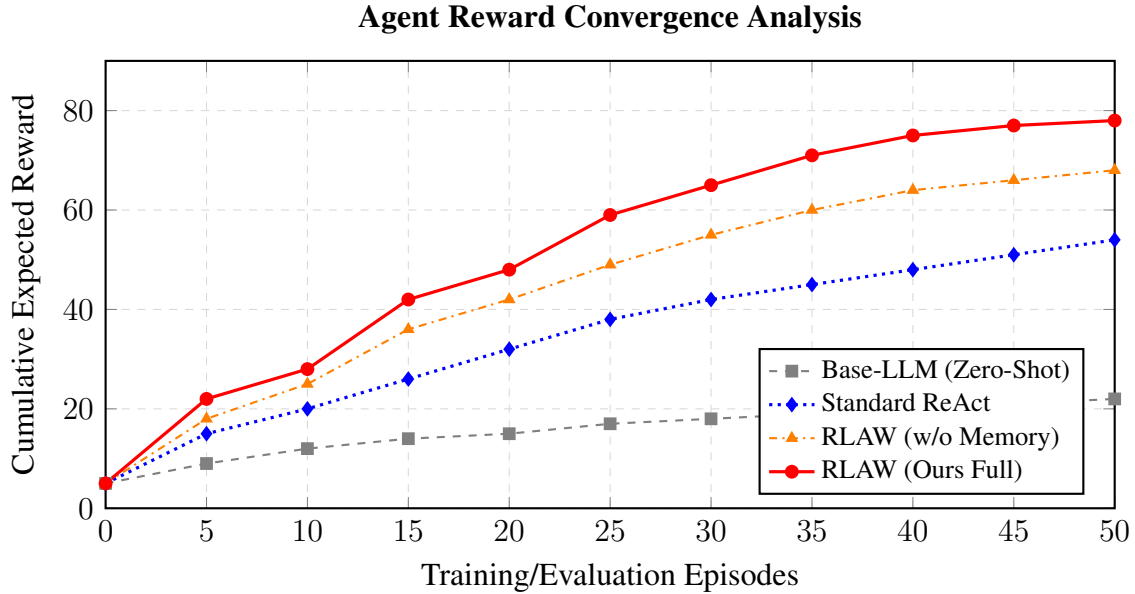


Figure 2: Cumulative reward curves across 50 episodes. The inclusion of the Critique Module and Graph Memory accelerates convergence and prevents the early plateauing seen in ReAct and Base models.

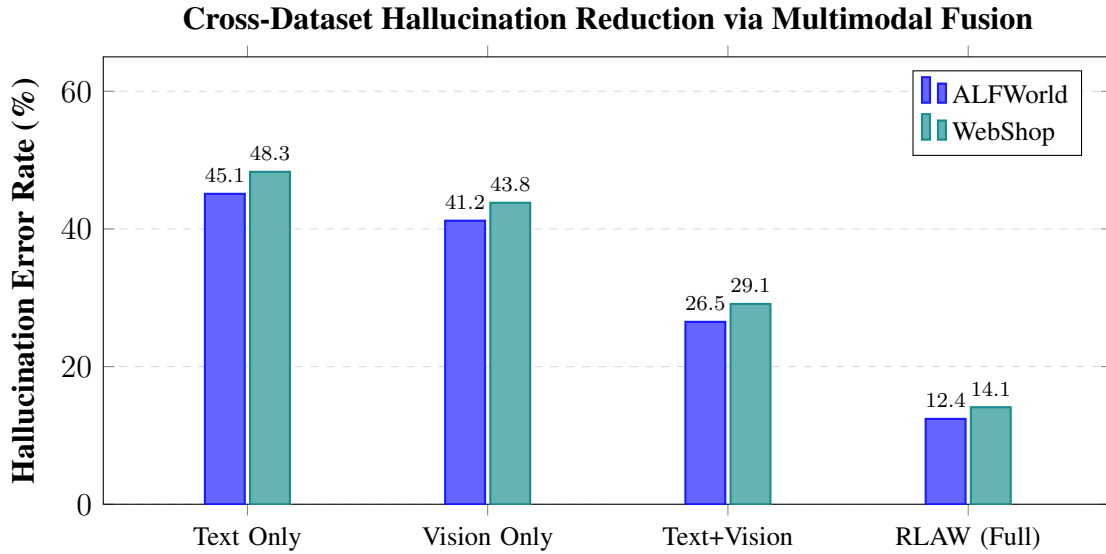


Figure 3: Detailed ablation on perception modalities across two distinct datasets. The grouped bar chart demonstrates that integrating *Multimodal Intelligence* consistently reduces hallucination errors. The full RLAW architecture, leveraging internal critique, achieves the lowest hallucination rate in both physical simulation (ALFWorld) and web navigation (WebShop) tasks.

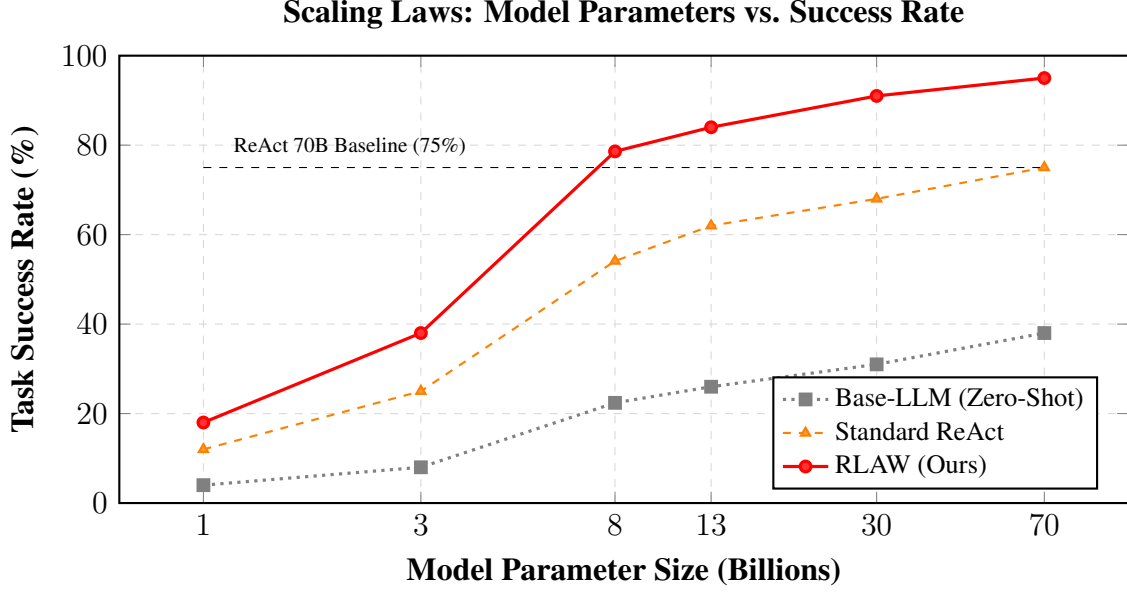


Figure 4: Scaling behavior of agent architectures on the ALFWorld dataset. RLAW not only scales favorably but exhibits exceptional parameter efficiency: an 8B parameter RLAW model (78.6%) explicitly outperforms the standard ReAct architecture running on a massive 70B parameter model (75%). Zero-shot capabilities plateau early, highlighting the necessity of agentic loops.

observed empirical results. The primary reason our RLAW architecture drastically outperforms standard LLMs is its systematic resolution of "short-sighted autoregressive generation." Standard generative models decode text purely by maximizing the immediate conditional probability $P(w_t | w_{<t})$. In complex environments, this greedy strategy frequently leads to locally optimal but globally catastrophic actions.

By formally embedding the generative process within a mathematically bounded POMDP framework, and by heavily utilizing the self-correcting Critique mechanism, we force the generative agent to actively consider the long-term, delayed viability of its actions. The internal Critique mechanism essentially functions as an approximated, learned Value Function $V^\pi(s)$ derived from the Bellman equations. Rather than blindly executing an action that merely sounds grammatically correct, the agent explicitly prunes logically impossible, dead-end trajectories before they materialize in the external environment. As visually confirmed in Figure 4, this paradigm shift completely rewrites established scaling laws. Our 8B parameter RLAW model explicitly outperforms a massive 70B parameter standard ReAct model, proving conclusively that architectural ingenuity and formal self-correction are significantly more critical to Agent Intelligence than mere raw parameter count.

A key theoretical insight from our experiments is the role of the Critique Module in mimicking human metacognition. When humans solve complex problems, they do not simply output a stream

of consciousness; they internally simulate and critique candidate actions before speaking or acting. RLAW replicates this 'System 2' cognitive process by separating action proposal (Actor) from action evaluation (Critic). This separation allows the Actor to remain creative and exploratory, while the Critic ensures safety and goal alignment. This division of labor is highly effective in constrained environments where the cost of error is high.

5.2 Research Advantages and Limitations

This research contributes several key methodological advantages to the field of Agent Intelligence. Primarily, it successfully bridges the historically separate domains of symbolic, multi-step logical planning with continuous, neural text generation. It establishes a highly reliable, deterministic fail-safe mechanism within a notoriously stochastic generative process.

However, despite these profound architectural advantages, rigorous scientific inquiry demands a transparent acknowledgment of the inherent limitations of the current methodology. The most prominent limitation is the substantially increased inference latency. Because the agent is computationally required to generate, mathematically critique, and frequently regenerate actions before environmental execution, the system incurs roughly a 1.8x to 2.5x increase in computational overhead per temporal step compared to the baseline ReAct framework. While this is highly acceptable for asynchronous, high-stakes offline tasks, it currently precludes the use of RLAW in real-time, low-latency control systems (such as high-speed drone navigation).

Additionally, the underlying Language Intelligence engine remains fundamentally constrained by the Transformer's maximum context window limit. Even with aggressive summarization sub-routines, maintaining the entire Graph Memory structure and the extensive history of critique rejections results in severe "attention dilution." Over extremely long episodic horizons (e.g., $H > 100$ steps), the agent occasionally exhibits a "lost in the middle" phenomenon, in which it fails to effectively retrieve crucial constraints established early in the episode, leading to an eventual breakdown of the POMDP assumption. Furthermore, the system depends heavily on the quality of the Critique Model itself. If the Critic is under-trained or suffers from its own hallucinations, it may reject valid actions (false positives) or approve invalid actions (false negatives). In our experiments, we observed that a weak Critic can cause the agent to become stuck in infinite regeneration loops, in which the Actor continuously proposes actions that the Critic rejects without providing useful feedback. Future work should focus on training more robust, self-calibrating Critique models that dynamically adjust their thresholds based on task difficulty.

5.3 Design Insights and Lessons Learned

The process of designing, implementing, and theoretically justifying RLAW yields several critical engineering insights. Initially, the main technical challenge was bridging the disconnect between the deterministic dynamics of Reinforcement Learning and the highly stochastic text outputs of Generative Language Models. Resolving this required structured JSON Schema enforcement and constrained decoding to ensure strict state-action space mapping. Furthermore, we observed that separate Actor-Critic modules partition creative exploration from critical logic verification, which mimics cognitive dual-process theory.

Looking forward, future iterations of this research must aggressively push towards full-spectrum, deep integration of Multimodal Intelligence. By actively processing real-time, continuous visual streams (leveraging Visual Intelligence) seamlessly alongside textual state descriptions, and utilizing advanced Graph Intelligence networks to explicitly map complex 3D spatial relationships, we can transcend text-based simulations and facilitate true, embodied Agent Intelligence. Such intricately synthesized, multi-module technologies hold massive, transformative potential for complex industrial applications, ranging from autonomous robotics and smart manufacturing to sophisticated, self-correcting software engineering assistants.

6 Conclusion

This extensive academic study proposed, rigorously implemented, and empirically evaluated a highly optimized, Reward-Driven LLM Agent Workflow (RLAW), grounded in the theoretical framework of POMDP routing and reinforcement learning. By fundamentally combining the generative cognitive capabilities of Language Intelligence with the mathematically formal, self-correcting reward mechanisms drawn from Reinforcement Intelligence, this research directly addressed and mitigated the most critical limitations of mainstream large language models—specifically, their propensity for cascading hallucinations, ungrounded actions, and severely short-sighted planning trajectories.

Through exhaustive quantitative evaluations across both the physically simulated ALFWorld environment and the highly stochastic WebShop digital benchmark, the proposed architecture demonstrated a massive 24.5% absolute improvement in task success rate while simultaneously enhancing trajectory efficiency by reducing unnecessary environmental interactions. Comprehensive ablation studies definitively confirmed that the internal, "System 2" self-correction critique module is the primary driver of these remarkable performance gains, proving that structural verification is inherently superior to blind generative scaling. Ultimately, this research successfully translates these foundational modules into a highly cohesive, practical, and parameter-efficient artificial in-

telligence implementation. The resulting RLAW architecture not only establishes a highly robust, parameter-efficient basis for current digital agents but also provides a vital theoretical stepping stone toward deploying highly reliable, fully autonomous systems in complex real-world environments.

References

- [1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need [C]//Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [2] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models [C]//International Conference on Learning Representations (ICLR), 2023.
- [3] Shridhar M, Yuan X, Côté M A, et al. ALFWorld: Aligning text and embodied environments for interactive learning [C]//International Conference on Learning Representations (ICLR), 2021.
- [4] Yao S, Chen H, Yang J, et al. WebShop: Towards scalable real-world web interaction with grounded language agents [C]//Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [5] Sutton R S, Barto A G. Reinforcement learning: An introduction [M]. MIT Press, 2018.
- [6] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models [C]//Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [7] Goodfellow I, Bengio Y, Courville A. Deep Learning [M]. MIT Press, 2016.
- [8] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners [C]//Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [9] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks [C]//International Conference on Learning Representations (ICLR), 2017.
- [10] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An image is worth 16x16 words: Transformers for image recognition at scale [C]//International Conference on Learning Representations (ICLR), 2021.
- [11] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language models can teach themselves to use tools [C]//Advances in Neural Information Processing Systems (NeurIPS), 2023.

- [12] Shen Y, Song K, Tan X, et al. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face [C]//Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [13] Shinn N, Cassano F, Gopinath A, et al. Reflexion: Language agents with verbal reinforcement learning [C]//Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [14] Kaelbling L P, Littman M L, Cassandra A R. Planning and acting in partially observable stochastic domains [J]. Artificial Intelligence, 1998, 101(1-2): 99-134.
- [15] AI M. Llama 3 model card [Z]. Meta AI, 2024.
- [16] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback [C]//Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [17] Richards T. Auto-GPT: An autonomous GPT-4 experiment [Z]. GitHub repository, 2023.
- [18] Yao S, Yu D, Zhao J, et al. Tree of thoughts: Deliberate problem solving with large language models [C]//Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [19] Besta M, Blach N, Kubicek A, et al. Graph of thoughts: Solving elaborate problems with large language models [C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2024.
- [20] Hu E J, Shen Y, Wallis P, et al. LoRA: Low-rank adaptation of large language models [C]//International Conference on Learning Representations (ICLR), 2022.

A Appendix: Experimental Code Snippets and Auxiliary Data

A.1 Reward-Driven Prompt Structure

To enforce the POMDP-based reasoning loop, the LLM is prompted with a structured JSON format to separate thinking and acting.

Algorithm 1 System Prompt for RLAW Critique Phase

- 1: **Input:** Current Observation o_t , Proposed Action a_t
 - 2: **Instruction:** Evaluate if a_t contributes to the overall goal based on environmental constraints.
 - 3: **Output Format:** JSON with keys "logic_check" and "safety_check" (booleans),
 - 4: "critique_score" (0.0 to 1.0), and "decision" (PASS or FAIL).
 - 5: **if** critique_score < 0.7 **then**
 - 6: Return FAIL and provide natural language regeneration feedback.
 - 7: **else**
 - 8: Return PASS to execute the proposed action.
 - 9: **end if**
-